RAG-Based Customer Support Assistant - High Level Design

System Overview:

• Problem Definition: Traditional customer support systems rely on static FAQs or unguided LLMs that often "hallucinate" or provide generic information not specific to a company's unique products.

• Scope of the System: The system provides an end-to-end pipeline to ingest PDF documents,store them as searchable embeddings, and use a graph-based workflow to answer user queries or escalate them to a human when necessary.

Architecture Diagram (Components) :

• **User Interface:** Streamlit UI for seamless interaction.

• **Document Ingestion Pipeline:** A modular script that automates the transition from raw PDF files to structured vector data.

• **Embedding System**: Converts text chunks into high-dimensional numerical vectors to enable semantic search.

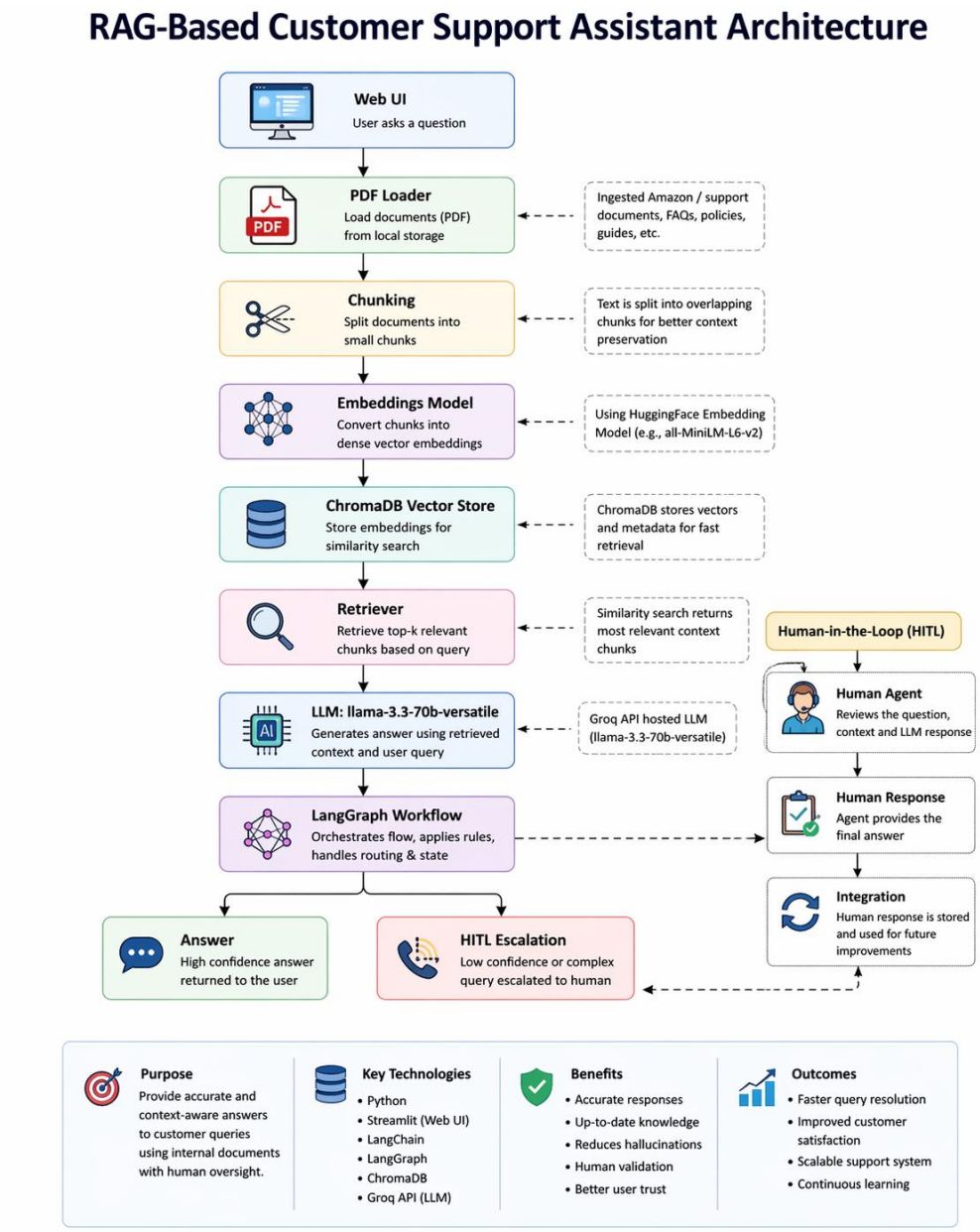
• **Vector Database:** A persistent storage layer (ChromaDB) that holds document embeddings and metadata.

• **Retrieval Layer:** The interface that queries the vector database to find the most relevant chunks based on a user's query.

• **LLM Processing Layer:** The intelligence layer (llama-3.3-70b-versatile via Groq) that synthesizes retrieved context into human-like responses.

• **Workflow Orchestration (LangGraph):** The "brain" that manages state transitions and directs data flow between processing nodes.

• **HITL System:** A safety-critical module that pauses the AI and requests human intervention for complex or out-of-scope queries.



Component Description:

• **Document Loader:** Uses PyPDFLoader to extract raw text and metadata from PDFknowledgebases.

• **Chunking Strategy:** Employs a RecursiveCharacterTextSplitter with a chunk size of 500 characters and 50-character overlap to preserve semantic context.

• **Embedding Model:** Utilizes the all-MiniLM-L6-v2 model from HuggingFace for efficient and accurate local vector generation.

• **Vector Store:** ChromaDB is used to store and index vectors, allowing for rapid similarity searches.

• **Retriever**: A similarity-based search mechanism that fetches the top K (e.g., $K=3$) most relevant document chunks.

• **LLM:** The Llama 3 model (hosted on Groq) provides high-speed inference and excellent instruction-following capabilities for customer support.

• **Graph Workflow Engine:** LangGraph orchestrates the system as a state machine, ensuring predictable and controllable AI behavior.

• **Routing Layer:** A conditional logic gate that analyzes the LLM's confidence and the quality of retrieved context to decide the next path.

• **HITL Module:** A terminal-based interruption node that facilitates direct human expert input into the conversation state.

Data Flow

• **How Data Moves from PDF → Answer:**

1. Ingestion: PDF is loaded, chunked, and embedded into ChromaDB.

2. Query: User submits a question through the FastAPI endpoint.

3. Retrieval: The system searches ChromaDB for matching text chunks.

4. Generation: LLM processes the query + context and generates a response.

5. Output: The final answer is returned to the user or escalated to a human.

• **Query Lifecycle:** Every query passes through a state-controlled cycle where it is either answered by the AI or flagged for human review based on predefined escalation criteria.

5. Technology Choices

• **Why ChromaDB:** It is an open-source, lightweight vector database that provides persistent storage without the need for complex external infrastructure.

• **Why LangGraph**: Unlike linear chains, LangGraph allows for cycles and humanin-the-loop interruptions, which are essential for production-grade decision systems.

• **LLM Choice:** Llama 3 via Groq was chosen for its best-in-class performance-tolatency ratio, ensuring near-instant support responses.

• Additional Tools: python-dotenv for security (API key management) .

6. Scalability Considerations

• **Handling Large Documents:** The system uses optimized chunking and vector indexing to ensure that adding more PDFs does not exponentially increase retrieval time.

• **Increasing Query Load:** API's asynchronous nature allows the system to handle multiple concurrent user requests efficiently.

• **Latency Concerns:** By using Groq for LLM inference and a local ChromaDB instance, we minimize network hops and ensure sub-second response times for most queries.